Batching semantic operators in Kalypso: when it pays, and how to choose block sizes

Batching study and cost model v3 · L40S, vLLM 0.29.0, Qwen2.5-7B-Instruct · full write-up: `results/analysis/STUDY.md` (PR #20)

Summary. A semantic operator (filter, join or map) can run one row per LLM call (*per-tuple*) or *k* rows per call (a *block*). We measured blocking for each operator alone and in two-stage pipelines (45 runs, 248 arms), fitted a latency cost model, and used it to choose each stage's block size. On 5 held-out experiments the algorithm was **1.38x** faster than per-tuple. The fastest arm measured was 1.45x, so the algorithm was 6.1% slower than the best possible on average. It beat every fixed block size (k=4: 1.08x, k=16: 0.69x, k=32: 0.47x) and beat tuning each operator on its own (1.31x).

1. Hardware and workloads

Item	Setting	Why it matters for batching
GPU	1× NVIDIA L40S 48 GB (Modal), one container per experiment	Sets prefill speed and decode-step time
Server	vLLM 0.29.0 OpenAI server, defaults (bf16 weights)	Batches separate requests on its own (continuous batching)
Batch limits	`max_num_seqs` 256, `max_num_batched_tokens` 2048, `max_model_len` 32768	256 calls run at once; more calls run in waves
Prefix cache	On, reset before each trial	Separate calls reuse shared prompt text, so per-tuple is already cheap
Model	Qwen2.5-7B-Instruct, temperature 0	Repeatable answers
Measure	1 warm-up + 3 repeats, median; arms interleaved A,B,A,B	Drift hits all arms equally

Workload	Op	Rows	Tokens/row
SemBench movie reviews: “clearly positive?”, true selectivity 7–92%	filter	100–1865	~31
Review pairs: “opposite sentiment?”, match rate 10–90%	join	16^2–64^2	~31/side
Reviews: “main reason, one sentence”	map	100–500	~31
ContractNLI: “contract entails hypothesis?”	filter	400	~1500
Contract × clause	join	24×8	~1600 + 15
filter→join, filter→map, map→filter	pipe	50–1000	~31

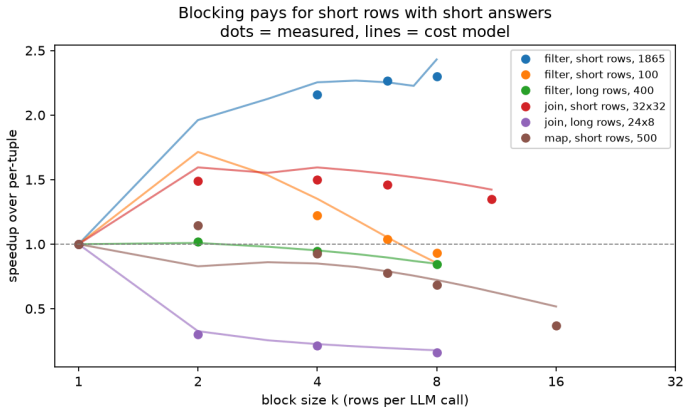
Correct answers come from dataset labels (SemBench sentiment, ContractNLI entailment). Maps have no labels, so they are scored by ROUGE-1 against per-tuple output. Rows over 512 tokens count as **long**.

2. Findings: operators alone and in pipelines

When does a block beat separate requests through vLLM?

vLLM already batches separate requests and caches shared prompt text. A block saves only the work each call repeats (instructions, answer format, extra waves). In exchange, one call has to write *k* answers one after another.

Case	Block?	Measured
Short rows, short answers (filter, join), many rows	Yes, k 2–8	filter 2.2x (1500 rows, k 6–7); join 1.5x (k 2–3)
Few rows (50–100)	Barely	1.0–1.1x; operator startup dominates
Long rows (~1500 tok)	No	≤1.05x; reading the row dominates. Recall 0.70→0.45
Long answers (map)	No	0.37–0.9x; one call writes all k answers
Large blocks (k ≥ 16)	No	0.47–0.69x, and accuracy falls



Speedup over per-tuple vs block size, per operator and row length.

How should blocks move through filters that shrink rows and joins that expand them?

Observation	Evidence
Blocks stay full: rows are held until a block has k of them; only the last one is partial	100% fill in every pipeline run checked
A filter's block size changes the next stage's work	A blocked filter passes 1.2–4x more rows to the join (recall 0.63 → 0.79)
Block the expensive stage, not the cheap one before it	Filter per-tuple + join k=2 was fastest (1.45x)
Blocking a neighbour makes a per-tuple map cut off and rerun answers	12–56% of map rows rerun
Block joins repeat the left row in every pair	~3.3x prompt tokens on long left rows
Stages on one GPU add up	L ≈ 0.98 × Σ stage latencies

Rules: (1) choose all stages' block sizes together, because a filter's k sets how many rows the next stage gets (1.38x jointly vs 1.31x per stage); (2) keep blocks full; (3) group block-join pairs by left row (proposed); (4) fix map output-length estimation before blocking around maps (proposed).

filter→join	s10	s25	s50	s75	s90
True positives (of 100)	7	30	48	68	92
Passed, filter per-tuple	2	16	28	42	58
Passed, filter blocked	8	23	36	50	78

3. Algorithm: deciding whether to block and what size

No fixed rule works, so the model has a fixed shape whose constants are fitted on measured runs (weighted least squares on relative error). Latency of one stage with N items at block size k:

$L = g[op] + \alpha \cdot P + (\beta + tail[op]) \cdot W \cdot D + \gamma \cdot C \cdot D + \delta \cdot C \cdot p^2$

Term	Meaning	Fitted (L40S)
g	Fixed cost to start the operator	map 0.29 s; filter/join ≈ –0.05 s
α·P	Reading prompts: P = prompt tokens not served by the cache	0.79 s / 10k tok
β·W·D	Writing answers: W waves of ≤256 calls × D tokens per answer	24.8 ms / step (+11 ms map)
γ·C·D	Bigger running batch slows each token	0.1 ms / token
δ·C·p^2	Attention on long prompts (p in 1k tokens)	1.77 ms

Calls C = N for per-tuple and ⌈N/k⌉ for blocks. Answer length D grows with k: filter –2.6+4.5k, join –2.6+4.6k, map 63+14.7k tokens. Prompt tokens come from the real prompt template. Pipelines pass row counts from stage to stage: a filter passes N·(recall(k)·s + fpr(k)·(1–s)) for true selectivity s, a join keeps pairs the same way, and map reruns are added. Total L = 0.98·Σ stages.

Fitted on 45 runs (173 single-operator arms, 75 pipeline arms). **Error:** 4.3% single operator, 13% pipelines, 8.9% on held-out picks.

Algorithm (input: the pipeline's operators, row count and token lengths, a selectivity estimate):

1. For each stage, the largest allowed k: fits the context window and is no larger than the largest size measured for that operator and row length.
2. Try every combination of block sizes (k=1 means no blocking).
3. For each one, predict every stage's latency, passing on the rows each filter or join keeps.
4. Pick the lowest total predicted latency.
5. Among options within 2% of the best (the model's error), choose the smallest blocks.
6. Optional accuracy guard: cap k where measured F1 dropped by more than 0.02 (maps: ROUGE-1 ≥ 0.8).

Example (filter→join, 100×32 rows, true s = 0.68). The model predicts 10.9 s for filter per-tuple + join k=2; measured 10.4 s, 1.45x, which was the fastest arm. Blocking the filter passes more rows and makes the join slower.

At run time (proposed): re-estimate the pass rate as rows go by, $\hat{\pi} = (\text{passed} + a)/(\text{seen} + a + b)$, and re-plan the stages not yet started. Switch only if the gain is over 5%, and only if the same choice wins at $\hat{\pi} \pm 2\sigma$.

Compared with Trummer (arXiv 2510.08489): his model minimises token cost, so bigger blocks always cost less. On a local GPU, latency rises past k ≈ 8, so his rule would pick our worst sizes. His left×right join layout is worth adopting.

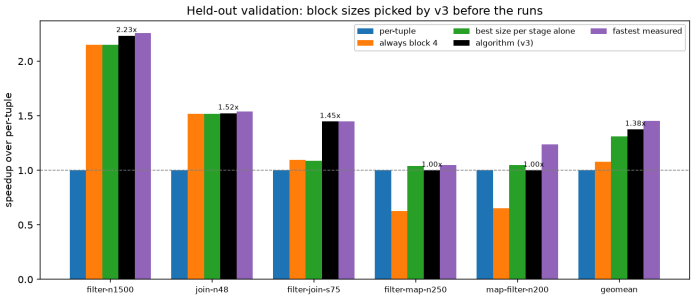
4. Validation experiments

Design	Setting
Held out	validate3: sizes, selectivities and shapes not in the fit. The model picks block sizes *before* the run (picks frozen in `algorithm_choices.json`)
Experiments	filter 1500 rows; join 48×48; filter→join s75; filter→map 250; map→filter 200
Strategies	Per-tuple; fixed k 4 / 16 / 32; best-in-isolation (each stage's best size alone); algorithm; algorithm + accuracy guard
Oracle	Fastest of all arms, plus a grid around the pick
Metrics	Speedup vs per-tuple (geometric mean), regret = slower than oracle, F1 / ROUGE-1 change

Strategy	Speedup	Regret	Kept acc.
Per-tuple	1.00x	50.6%	5/5
Fixed k=4	1.08x	39.0%	2/5
Fixed k=16	0.69x	162.8%	1/5
Fixed k=32	0.47x	346.4%	1/5
Best in isolation	1.31x	11.6%	3/5
Algorithm (v3)	**1.38x**	**6.1%**	**3/5**
Algorithm + guard	1.17x	25.6%	5/5
Oracle	1.45x	0%	3/5

Experiment	Pick	Speedup	Regret	ΔF1
Filter, 1500 rows	k 6	2.23x	1.1%	+0.11
Join, 48×48	k 2	1.52x	0.9%	–0.18
Filter→join, s75	1, 2	1.45x	0%	–0.13
Filter→map, 250	per-tuple	1.00x	4.7%	0.00
Map→filter, 200	per-tuple	1.00x	23.8%	0.00

The one miss is map→filter: the model predicted 5.3 s for per-tuple and it took 7.1 s. Repeats varied by about 1 s. Earlier round (picked by v2): 1.07x, regret 12.1%. v3 halved regret on unseen experiments.



Held-out speedups per strategy and experiment.

Next rounds (picks frozen), same baselines. (1) `validate-jf`, **join→filter**, 32×32 review pairs: the join keeps same-sentiment pairs (18–62%), then the filter keeps both-positive pairs (10–90%). Includes a 4×4 grid of join × filter sizes. Model picks join ≈10, filter 2, because a blocked join keeps fewer pairs. Guarded pick: join 1, filter 2–3. `validate4`: (2) **held-out long rows**: ContractNLI filter (75%, 300 rows) and join (40×8). Model picks per-tuple. (3) **Row-length sweep**: contracts cut to ~146 / 267 / 505 / 937 tokens per row. Model picks k=2 (per-tuple at 937 when guarded).

5. Limits

One GPU and one model: other hardware needs a refit on a smaller sweep, but the model's shape stays the same. The latency-first pick loses F1 on joins (–0.13 to –0.18), so use the guard when accuracy matters. Only two row lengths are measured so far (~30 and ~1500 tokens); the in-between lengths come with `validate4`. Untested: sending a partial block early instead of waiting for it to fill.